

🌐 Git 与 GitHub 速查

打印提示：A4、实际大小（100%）、关闭浏览器“页眉和页脚”；需要保留彩色底色时开启“背景图形”。用法：第 1 页背概念，第 2 页是每天要用的命令，第 3~4 页在卡住时翻。

目录

第 1 页	三个概念 + 首次配置（一劳永逸）
第 2 页	最小工作流：六条命令走完一个循环
第 3 页	撤销、查看、分支（出错时翻）
第 4 页	.gitignore + 常见报错 + 提交规范 + 小练习

第 1 页 · 三个概念与首次配置

一句话理解：Git = 给你的代码**拍快照**（随时能回到过去）；GitHub = 把快照**存到云端**（换电脑不丢、别人能看）。

找工作时，面试官看的就是你的 GitHub 仓库——**提交记录 = 你最真实的学习履历**。

1.1 三个概念（记住这个比喻就够）

概念	比喻	对应命令
仓库 (repository)	一本带“存档功能”的相册	git init
提交 (commit)	按下一次“存档”：记录这一刻的所有文件	git add + git commit
远程 (remote)	云端备份（GitHub 上的那本相册）	git push / git pull

工作区 → (add) → 暂存区 → (commit) → 本地仓库 → (push) → GitHub。四个格子，命令只在格子之间搬东西。

1.2 首次配置（只做一次，五条命令）

```
$ git --version                # 没装就先: sudo apt install git
$ git config --global user.name "你的名字"
$ git config --global user.email "你的邮箱"    # 建议用 GitHub 注册邮箱
$ ssh-keygen -t ed25519        # 一路回车，生成 SSH 钥匙
$ cat ~/.ssh/id_ed25519.pub    # 复制这一整行 → 粘到 GitHub
```

粘贴位置：GitHub 网页 → 右上角头像 → **Settings** → **SSH and GPG keys** → New SSH key → 粘贴 → 保存。

验证：ssh -T git@github.com 出现 Hi 你的用户名！就成了。

第 2 页 · 最小工作流：一个循环六条命令

日常循环（改代码 → 存档 → 上传），背下来：

```
$ git status          # ① 看：哪些文件改了（最常用，随时敲）
$ git add .           # ② 挑：把改动放进暂存区（. = 全部）
$ git commit -m "说明这次改了什么" # ③ 存：拍快照
$ git push            # ④ 传：同步到 GitHub
```

2.1 第一次：把本地目录变成仓库并推上去

```
$ cd ~/lab/myproject
$ git init          # 变成 Git 仓库（多出一个 .git 隐藏目录）
$ git add . && git commit -m "首次提交"
$ git branch -M main # 主分支命名为 main（GitHub 的默认叫法）
$ git remote add origin git@github.com:你的用户名/仓库名.git
$ git push -u origin main # 第一次要 -u，以后直接 git push
```

先在 GitHub 网页上点 **New repository** 建一个空仓库（**不要**勾选 "Add a README"，否则两边不一致会推不上去），再把上面的地址粘过来。

2.2 看一眼历史

```
$ git log --oneline # 一行一条提交记录（按 q 退出）
$ git diff          # 看还没暂存的改动（改了哪几行）
```

第 3 页 • 撤销、查看、分支 (出错时翻)

🔄 改错了想撤回 (按"改到哪一步"选)

情况	命令	说明
改了文件、还没 add	<code>git restore 文件名</code>	丢弃工作区改动, 不可找回
已 add、还没 commit	<code>git restore --staged 文件名</code>	从暂存区拿出来, 改动仍在
已 commit、想改说明	<code>git commit --amend -m "新说明"</code>	只改最近一次 (还没 push 时用)
想回到某次提交	<code>git log --oneline</code>	先找到那次的编号 (前 7 位), 再决定怎么处理
想丢弃全部本地改动	<code>git reset --hard</code>	危险 : 所有未提交的改动全没

⚠ 记住一条: **没提交的东西, Git 救不回来**。所以养成"小步提交"的习惯——改一点、存一次。

🌿 分支 (先知道, 别急着用)

```
$ git branch          # 看有哪些分支 (* 是当前)
$ git checkout -b feature-login  # 新建并切到新分支 (试新想法不污染主线)
$ git checkout main    # 切回主线
$ git merge feature-login  # 把分支合并回主线
```

🔄 多人/多机同改: 先拉后推

```
$ git pull          # 开工前: 把云端最新改动拉下来
$ git push          # 收工前: 把自己的改动推上去
```

口诀: **开工先 pull, 收工再 push**。两台机器 (或和同事) 同时改, 就不会打结。

🕒 .gitignore: 哪些文件不提交

在仓库根目录建一个名为 .gitignore 的文件，写上不要提交的东西（每行一条）：

```
# 依赖与缓存（体积大、能重建）
.venv/
__pycache__/
*.pyc

# 日志与临时文件
*.log
tmp/

# 含密码的配置（千万别传！）
*.env
config_secret.*
```

📌 记住：能自动重建的东西不提交（.venv、__pycache__），要重建它的清单要提交（requirements.txt）。

🚗 五个最常见的报错（人话版）

报错	意思	怎么办
fatal: not a git repository	当前目录不是 Git 仓库	进到仓库目录；或先 git init
nothing to commit, working tree clean	没有新改动可提交	不是错误：你没改东西，或已经提交过了
failed to push some refs	云端有你本地没有的提交	先 git pull，再 git push
Permission denied (publickey)	GitHub 不认你的钥匙	重新把 ~/.ssh/id_ed25519.pub 贴到 GitHub；或测试 ssh -T git@github.com
CONFLICT (content): Merge conflict	两边改了同一行	打开文件找 <<<<<<< 标记，人工留一份，然后 add + commit

✍ 提交信息怎么写（面试官真的会看）

写法	示例	点评
✗ 太随意	update / 改了一下 / aaa	看不出做了什么
✅ 推荐	新增磁盘巡检脚本 / 修复日志解析的空行报错	一句话说清“做了什么”
✅ 进阶（可选）	feat: 磁盘巡检脚本 / fix: 日志解析空行	加前缀：feat 新功能、fix 修复、docs 文档

🎯 小练习（做完你的 GitHub 就有内容了）

① 把 ~/lab 下写好的脚本目录变成 Git 仓库；② 建好 .gitignore（按上面四类）；③ 提交 3 次以上（每次说明写清楚）；④ 推到 GitHub；⑤ 在网页上打开仓库，确认能看到文件和历史记录。

📌 检查点对接：学习计划的“第 2 周末 commit ≥ 10 次”“第 4 周末脚本进 GitHub”——就靠这一册完成。